

실험 환경 및 소스코드

운영체제 (OS): 우분투

하드웨어 (Device): 아수스 제피러스 G14

Github:

<https://github.com/1st0Groom/university/tree/main/opencv/>

사진 : KTWIZ 안현민 선수

원본 이미지



(이미지와 R, G, B 각 채널별 히스토그램 출력 결과)

```
#1. 히스토그램 구하기 (프로그램 3-2)
import cv2 as cv
import matplotlib.pyplot as plt
import os

# 파일 경로 설정
script_dir = os.path.dirname(os.path.abspath(__file__))
img_path = os.path.join(script_dir, 'baseball.jpg')
img = cv.imread(img_path)

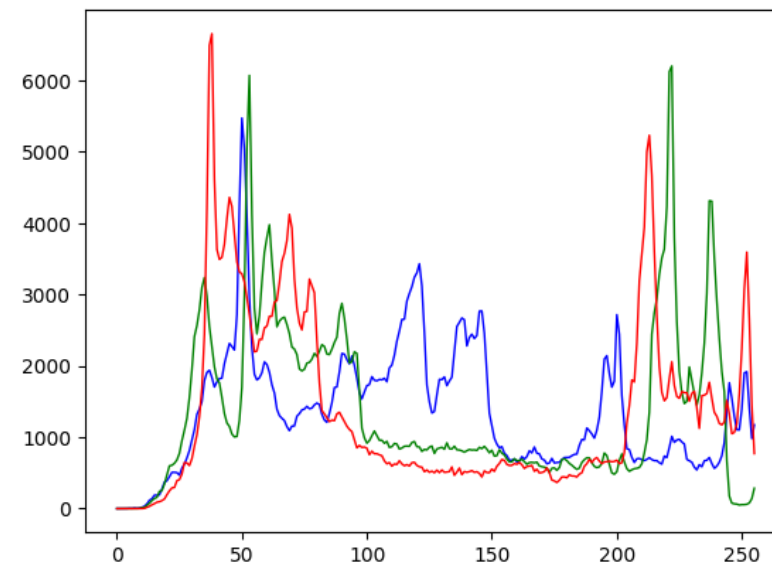
# B, G, R 채널 분리
b = img[:, :, 0]
g = img[:, :, 1]
r = img[:, :, 2]

# 각 채널별로 히스토그램을 계산하고 그리기
h_b = cv.calcHist([b], [0], None, [256], [0, 256])
h_g = cv.calcHist([g], [0], None, [256], [0, 256])
h_r = cv.calcHist([r], [0], None, [256], [0, 256])

# matplotlib을 이용하여 히스토그램 그리기
plt.plot(h_b, color='b', linewidth=1)
plt.plot(h_g, color='g', linewidth=1)
plt.plot(h_r, color='r', linewidth=1)

cv.imshow('Original', img)
plt.show()
```

히스토그램



02 - 1 색체계 변환(RGB to YUV 변환)

(Y, U, V 채널별 분리 결과)

```
# 색체계 변환
import cv2 as cv
import sys
import os

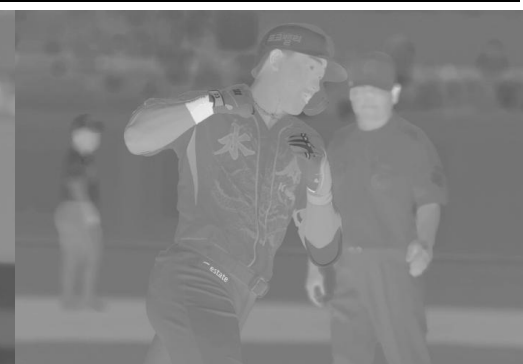
# 파일 경로 설정
script_dir = os.path.dirname(os.path.abspath(__file__))
img_path = os.path.join(script_dir, 'baseball.jpg')
img = cv.imread(img_path)

if img is None:
    sys.exit('이미지를 불러올 수 없습니다.')

yuv=cv.cvtColor(img, cv.COLOR_BGR2YUV)
y=cv.split(yuv)[0]
u=cv.split(yuv)[1]
v=cv.split(yuv)[2]

cv.imshow('YUV image', yuv)
cv.imshow('Y image', y)
cv.imshow('U image', u)
cv.imshow('V image', v)

cv.waitKey()
cv.destroyAllWindows()
```

YUV (전체)**Y 채널****U 채널****V 채널**

02 - 2 색체계 변환(RGB to YCbCr 변환)

(Y, Cb, Cr 채널별 분리 결과)

```
# 색 체계 변환 2
import cv2 as cv
import sys
import os

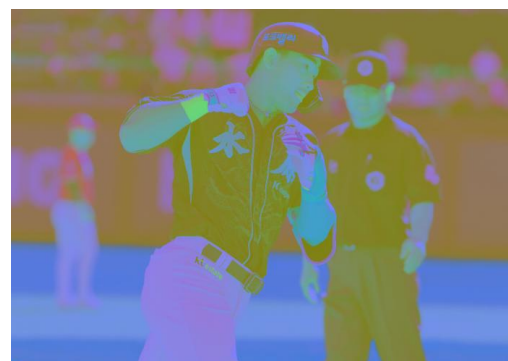
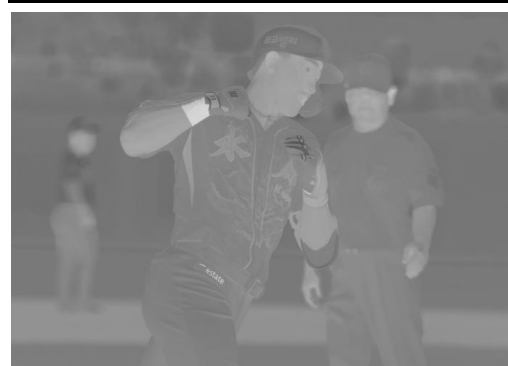
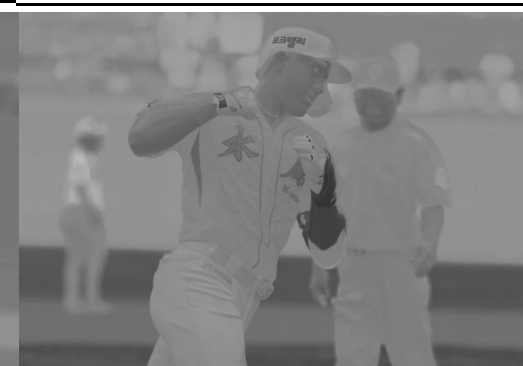
# 파일 경로 설정
script_dir = os.path.dirname(os.path.abspath(__file__))
img_path = os.path.join(script_dir, 'baseball.jpg')
img = cv.imread(img_path)

if img is None:
    sys.exit('이미지를 불러올 수 없습니다.')

yCbCr=cv.cvtColor(img, cv.COLOR_BGR2YCrCb)
y=cv.split(yCbCr)[0]
Cb=cv.split(yCbCr)[1]
Cr=cv.split(yCbCr)[2]

cv.imshow('yCbCr image', yCbCr)
cv.imshow('y image', y)
cv.imshow('Cb image', Cb)
cv.imshow('Cr image', Cr)

cv.waitKey()
cv.destroyAllWindows()
```

YUV (전체)**Y 채널****Cb 채널****Cr 채널**

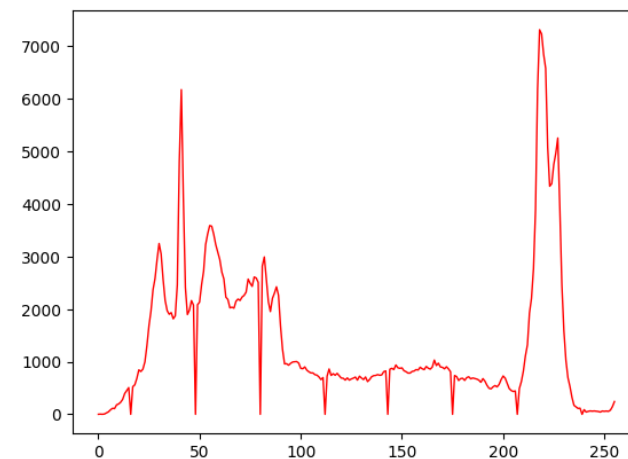
명암 대비 스트레칭

비교 분석 : 좁은 구역에 몰려있던 픽셀 값들을 0부터 255까지 선형적으로 잡아당겨 늘렸습니다. 히스토그램의 형태 자체는 유지되면서 양옆으로 넓게 퍼진 것을 확인할 수 있으며, 이로 인해 원본보다 대비가 적당히 증가하여 선명해진 결과를 보여줍니다.

결과 이미지



히스토그램



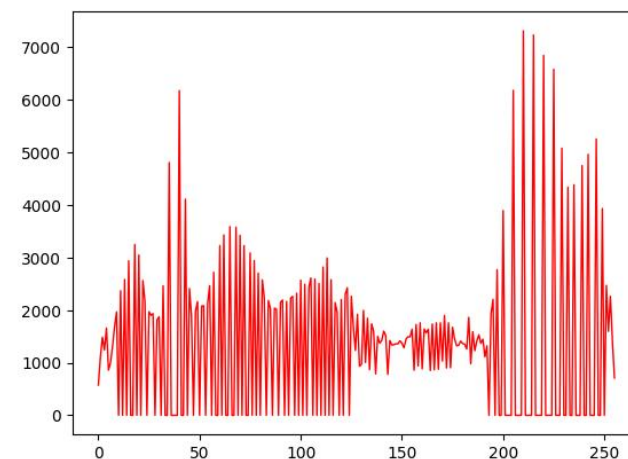
히스토그램 평활화

비교 분석 : 히스토그램을 단순히 늘리는 것을 넘어, 픽셀의 분포가 모든 밝기 영역에서 최대한 균일해지도록 강제로 재배치했습니다. 스트레칭보다 훨씬 더 강력한 대비 향상을 가져오지만, 픽셀이 재분배되면서 일부 영역에 과보정이 일어나거나 노이즈가 강조되는 현상을 볼 수 있습니다.

결과 이미지



히스토그램



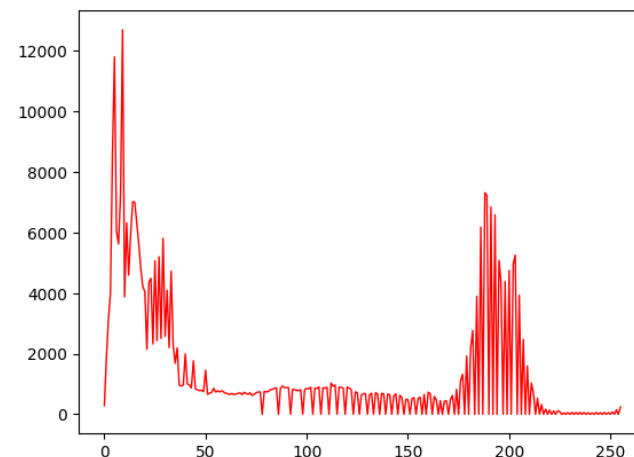
감마 보정

비교 분석 : 비선형적인 수식을 통해 픽셀의 밝기를 조정했습니다.
적용한 감마 값에 따라 히스토그램의 전체적인 픽셀 분포가 왼쪽 (어두운 쪽) 또는 오른쪽(밝은 쪽)으로 쏠리면서 원본보다 이미지가 전체적으로 어두워지거나 밝아지는 효과를 확인할 수 있습니다.

결과 이미지



히스토그램




```
# 명암 대비 스트레칭
```

```
import cv2 as cv
import os
import matplotlib.pyplot as plt
import sys
```

```
# 파일 경로 설정
```

```
script_dir = os.path.dirname(os.path.abspath(__file__))
img_path = os.path.join(script_dir, 'baseball.jpg')
img = cv.imread(img_path)
```

```
if img is None:
    sys.exit('이미지를 불러올 수 없습니다.')
```

```
#명암 조절할 때엔 흑백으로 처리하는게 원리 이해에 쉽다.
```

```
gray = cv.imread(img_path, cv.IMREAD_GRAYSCALE)
```

```
# minmax 정규화 -> 255에 맞춰 늘림 (첫 번째:원본이미지,두 번째:결과이미지배열,세 번째:
최소값, 네 번째:최대값, 다섯 번째: 정규화 방식)
```

```
stretch=cv.normalize(gray,None,0,255,cv.NORM_MINMAX)
```

```
# 히스토그램 계산
```

```
h_stretch = cv.calcHist([stretch], [0], None, [256], [0,256])
```

```
# 히스토그램 그리기
```

```
plt.plot(h_stretch, color='r', linewidth=1)
plt.show()
cv.imshow('stretch', stretch)
cv.waitKey(0)
cv.destroyAllWindows()
```

```
# 히스토그램 평활화
```

```
import cv2 as cv
import os
import matplotlib.pyplot as plt
import sys
```

```
# 파일 경로 설정
```

```
script_dir = os.path.dirname(os.path.abspath(__file__))
img_path = os.path.join(script_dir, 'baseball.jpg')
img = cv.imread(img_path)
```

```
if img is None:
    sys.exit('이미지를 불러올 수 없습니다.')
```

```
#명암 조절할 때엔 흑백으로 처리하는게 원리 이해에 쉽다.
```

```
gray = cv.imread(img_path, cv.IMREAD_GRAYSCALE)
```

```
# 평활화
```

```
hequalize=cv.equalizeHist(gray)
```

```
# 히스토그램 계산
```

```
h_equalize = cv.calcHist([hequalize], [0], None, [256], [0,256])
```

```
# 히스토그램 그리기
```

```
plt.plot(h_equalize, color='r', linewidth=1)
plt.show()
cv.imshow('equalize', hequalize)
cv.waitKey(0)
cv.destroyAllWindows()
```



```

# 감마보정
import numpy as np
import cv2 as cv
import os
import matplotlib.pyplot as plt
import sys

# 파일 경로 설정
script_dir = os.path.dirname(os.path.abspath(__file__))
img_path = os.path.join(script_dir, 'baseball.jpg')
img = cv.imread(img_path)

if img is None:
    sys.exit('이미지를 불러올 수 없습니다.')

# 흑백으로 불러오기
gray = cv.imread(img_path, cv.IMREAD_GRAYSCALE)

# 감마보정
gamma = 0.5

#수학공식 활용
#감마 역수(역수를 사용하는 이유: CG에서 감마 보정 공식은 원래 밝기의 (1/감마)^2 으로 정
의하기 때문)
invGamma = 1.0/ gamma
#i/255.0 : 수 줄이기, **invGamma:**는 제곱 연산, *255.0: 다시 256배 해주기, uint8는 0
부터 255까지의 정수를 담는 자료형
table = np.array([((i/255.0)**invGamma)*255.0 for i in
np.arange(0,256)]).astype('uint8')

#LUT(look-up table) 적용
gamma_img =cv.LUT(gray,table)

gamma_hist=cv.calcHist([gamma_img], [0], None, [256], [0,256])

# 히스토그램 그리기
plt.plot(gamma_hist, color='r', linewidth=1)
plt.show()
cv.imshow('gamma_img', gamma_img)
cv.waitKey(0)
cv.destroyAllWindows()

```

양선형 보간 그리고 Lanczos 보간

비교분석:

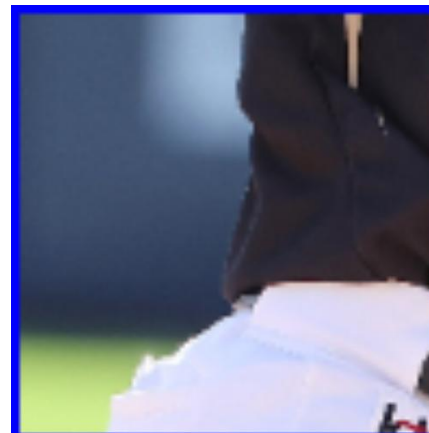
- 양선형 보간: 주변 4개의 픽셀을 거리 비율로 가중 평균내어 빈 공간을 채우는 방식으로, 보편적으로 가장 많이 사용됩니다. 계산 속도와 품질의 밸런스가 좋지만 이미지가 약간 흐려지는 현상이 나타납니다.
- Lanczos 보간: 더 넓은 주변 픽셀 영역을 참조하여 픽셀을 채우기 때문에 연산 속도는 느리나, 디테일을 보존하는 능력이 뛰어나, 양선형 보간보다 선명하고 품질이 높은 결과를 보여줍니다.

결과 이미지



양선형 보간 (Bilinear)

Lanczos 보간 (Lanczos4)



```
#양선형 보간
import cv2 as cv
import os
import sys

# 파일 경로 설정
script_dir = os.path.dirname(os.path.abspath(__file__))
img_path = os.path.join(script_dir, 'baseball.jpg')
img = cv.imread(img_path)

if img is None:
    sys.exit('이미지를 불러올 수 없습니다.')

patch = img[250:350, 170:270, :]

img = cv.rectangle(img, (170, 250), (270, 350), (255, 0, 0), 2)

양선형보간 = cv.resize(patch, dsize=(0, 0), fx=5, fy=5, interpolation=cv.INTER_LINEAR)
Lanczos보간 = cv.resize(patch, dsize=(0, 0), fx=5, fy=5, interpolation=cv.INTER_LANCZOS4)

cv.imshow('Original', img)
cv.imshow('Resize bilinear', 양선형보간)
cv.imshow('Resize lanczos4', Lanczos보간)

cv.waitKey()
cv.destroyAllWindows()
```

1차 미분 기반 Scharr, Canny

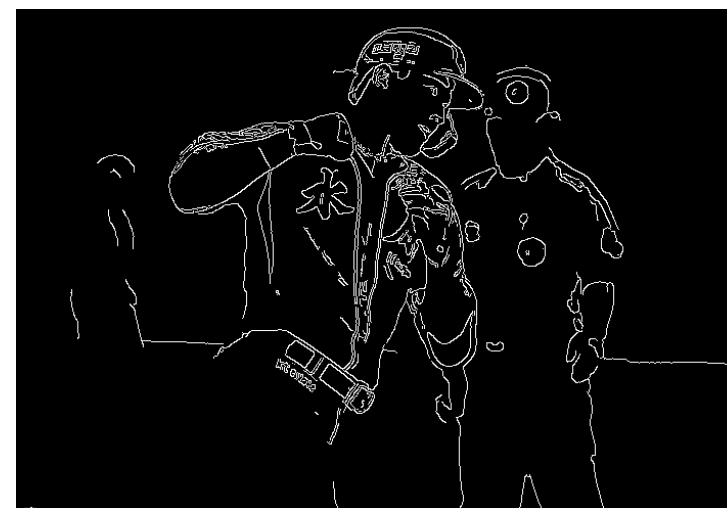
비교분석:

- Scharr: Sobel 알고리즘의 단점을 보완하여 미세한 방향 변화에 강합니다. X축과 Y축의 미분값을 합산한 결과로, 굵고 뚜렷한 윤곽선을 뽑아내지만, 다소 거칠고 노이즈가 섞일 수 있습니다.
- Canny: 노이즈 제거, 그래디언트 계산, 비최대 억제 등 다단계 과정을 거치는 가장 강력한 에지 검출기입니다. 다른 알고리즘에 비해 굵기가 적고 두께가 1픽셀 수준으로 아주 깔끔하고 정교한 윤곽선 결과를 보여줍니다.

Scharr 알고리즘



Canny 알고리즘

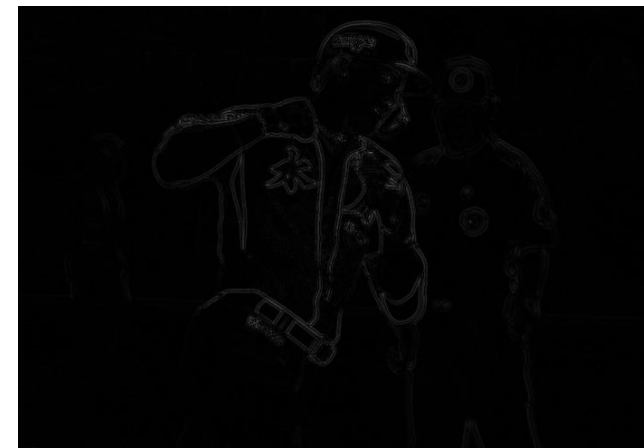


2차 미분 기반 LoG, DoG

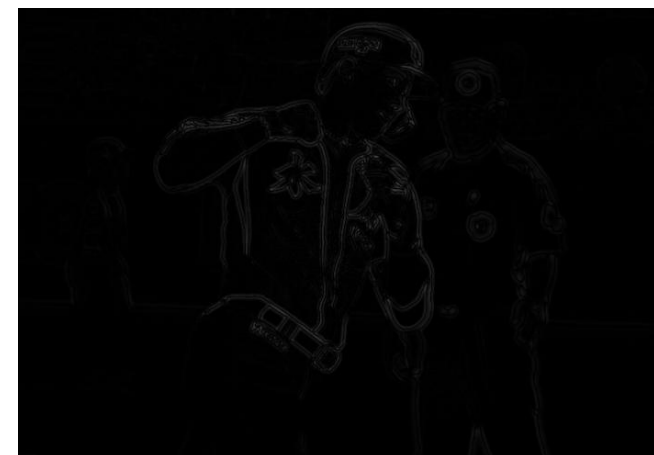
비교분석:

- LoG : 2차 미분인 라플라시안 필터가 노이즈에 매우 취약하다는 단점을 보완하기 위해 가우시안 블러를 먼저 적용한 결과입니다. 노이즈는 줄이면서도 에지를 안정적이고 부드럽게 찾아냅니다.
- DoG : 복잡한 2차 미분 연산 대신 흐린 정도가 다른 두 가우시안 블러 이미지의 단순 차이를 이용해 LoG를 근사한 결과입니다. 처리 속도가 훨씬 빠르면서도 LoG와 시각적으로 거의 유사한 훌륭한 에지 검출 성능을 보여줍니다.

LoG (Laplacian of Gaussian)



DoG (Difference of Gaussians)



```

# Scharr 알고리즘과 Canny 알고리즘
import cv2 as cv
import os
import sys

# 파일 경로 설정
script_dir = os.path.dirname(os.path.abspath(__file__))
img_path = os.path.join(script_dir, 'baseball.jpg')
img = cv.imread(img_path)

if img is None:
    sys.exit('이미지를 불러올 수 없습니다.')

# 그레이 스케일 변환
gray = cv.cvtColor(img, cv.COLOR_BGR2GRAY)

# --- 1. Scharr 알고리즘 ---
# x축 방향(가로선)과 y축 방향(세로선)의 미분을 따로따로 구한 뒤 합쳐서 더 정교한 에지를 얻어냅니다.

# 1단계: x방향 미분 (수직선 에지를 잘 찾음) -> 인자: (이미지, 8비트타입, x방향 1, y방향 0)
scharrx = cv.Scharr(gray, cv.CV_8U, 1, 0)

# 2단계: y방향 미분 (수평선 에지를 잘 찾음) -> 인자: (이미지, 8비트타입, x방향 0, y방향 1)
scarry = cv.Scharr(gray, cv.CV_8U, 0, 1)

# 3단계: 가로 방향과 세로 방향 에지를 50% 대 50% 비율로 예쁘게 합치기
scharr = cv.addWeighted(scharrx, 0.5, scarry, 0.5, 0)

# --- 2. Canny 알고리즘 ---
# 노이즈 제거 -> 미분 -> 얇은 선만 남기기 -> 선 연결하기 등 여러 단계를 거쳐서 아주 깔끔한 '1픽셀 두께'의 선명한 에지를 찾아냅니다.

# 인자: (이미지, 하위 임계값 100, 상위 임계값 200)
# 픽셀 변화량이 200 이상이면 무조건 에지로 판별, 100~200 사이이면 확실한 에지랑 연결되어 있을 때만 에지로 판별합니다.
canny = cv.Canny(gray, 100, 200)

cv.imshow('Original', img)
cv.imshow('Scharr', scharr)
cv.imshow('Canny', canny)

cv.waitKey(0)
cv.destroyAllWindows()

```

```

#LoG 알고리즘과 DoG 알고리즘
import cv2 as cv
import os
import sys

# 파일 경로 설정
script_dir = os.path.dirname(os.path.abspath(__file__))
img_path = os.path.join(script_dir, 'baseball.jpg')
img = cv.imread(img_path)

if img is None:
    sys.exit('이미지를 불러올 수 없습니다.')

# 에지 검출은 보통 흑백(Grayscale) 이미지로 수행합니다.
gray = cv.cvtColor(img, cv.COLOR_BGR2GRAY)

# Laplacian of Gaussian
# 1. 가우시안 블러로 노이즈 제거
blur = cv.GaussianBlur(gray, (5,5),0)

# 2. 라플라시안 필터 적용
# 2차 미분시 음수값 발생, 8비트에서 16비트로 계산해야함. 16은 bit, S는 양수부터 음수까지
LoG = cv.Laplacian(blur, cv.CV_16S)

# 3. 화면에 그리기 위해 절대값을 씌우고 다시 8비트로 변환
LoG = cv.convertScaleAbs(LoG)

# Difference of Gaussians
# 흐린 정도가 다른 가우시안 블러 이미지를 만들어서 둘을 뺌.
# 계산이 복잡한 LoG를 근사한 방법. 속도가 훨씬 빠름.

D1_blur = cv.GaussianBlur(gray, (5,5),sigmaX=1.0)
D2_blur = cv.GaussianBlur(gray, (9,9),sigmaX=2.0)

# 2개의 블러링 이미지의 차이를 절댓값으로 계산
DOG = cv.absdiff(D1_blur,D2_blur)

# 결과 확인
cv.imshow('Original Image',img)
cv.imshow('LoG Edges',LoG)
cv.imshow('DOG Edges', DOG)
cv.waitKey(0)
cv.destroyAllWindows()

```